Escola Profissional
BENTO DE JESUS CARAÇA
DELEGAÇÃO DE LISBOA

RELATÓRIO

RELATÓRIO DE PROJETO

1.º Trabalho Avaliativo

SISTEMA DE GESTÃO DE HOTEL

Programação Orientada a Objetos: herança, polimorfismo, classes abstratas e módulos

CURSO TÉCNICO DE PROGRAMAÇÃO E GESTÃO DE SISTEMAS INFORMÁTICOS

Docente: Professor Breno Sousa

Alunos:
Matheus Gimenes – Nº L2567
Dinis Monteiro – Nº L2557
Miguel Bancaleiro – Nº L2569

11/10/2026

Ciclo de Formação [2024/2027]
Ano Letivo [2026/2027]

ÍNDICE

INTRODUÇÃO.....1
CAPÍTULO I — ENQUADRAMENTO E CARACTERIZAÇÃO DO PROJETO.....2
1.1. Enquadramento.....2
1.2. O problema proposto.....2
1.3. Requisitos do sistema.....3
1.4. Ferramentas e recursos utilizados.....4
1.5. Critérios de avaliação.....4
CAPÍTULO II — ATIVIDADES DESENVOLVIDAS E JUSTIFICAÇÃO TÉCNICA.....6
2.1. Arquitetura geral da solução.....6
2.2. Módulo pessoa.py – a classe abstrata Pessoa.....7
2.3. Módulo quarto.py – Quarto, QuartoSimples e QuartoLuxo.....8
2.4. Módulo hospede.py – a classe Hospede.....10
2.5. Módulo funcionario.py – a hierarquia de funcionários.....12
2.6. Módulo tecnico_recepcao.py – herança múltipla.....14
2.7. Módulo main.py – o programa principal.....16
2.8. Conceitos de POO aplicados, com exemplos.....22
2.9. Testes e resultados.....27
2.10. Verificação face ao enunciado.....31
2.11. Dificuldades encontradas e soluções.....32
CAPÍTULO III — ANÁLISE DO PERCURSO PESSOAL.....33
3.1. Aprendizagens técnicas.....33
CAPÍTULO IV — CONCLUSÃO.....33

ÍNDICE DE FIGURAS

Figura 1 – Hierarquia de classes de pessoas (classes a tracejado são abstratas).....3
Figura 2 – Hierarquia de classes de quartos.....3
Figura 3 – Dependências (imports) entre os módulos do projeto.....6
Figura 4 – Associação entre Hospede e Quarto.....11
Figura 5 – Ordem de resolução de métodos (MRO) da classe TecnicoRecepcao.....15
Figura 6 – Fluxograma do menu principal.....22
Figura 7 – MRO e verificações isinstance de um TecnicoRecepcao (execução real).....23
Figura 8 – Polimorfismo: a mesma chamada, quatro comportamentos diferentes.....24
Figura 9 – Tentativa de instanciar classes abstratas (execução real).....24
Figura 10 – Validação com setters e associação hóspede–quarto (execução real).....25
Figura 11 – Execução: registo de hóspedes, contas e checkout.....28
Figura 12 – Execução: criação de funcionários e relatório do gerente.....30
Figura 13 – Execução: registo de reparos (polimorfismo).....31

INTRODUÇÃO

O presente relatório descreve o desenvolvimento do **Sistema de Gestão de Hotel**, realizado em grupo no âmbito do 1.º Trabalho Avaliativo proposto pelo Professor Breno Sousa, no Curso Técnico de Programação e Gestão de Sistemas Informáticos. O sistema foi escrito na linguagem **Python** e tem como objetivo principal demonstrar a aplicação correta dos pilares da programação orientada a objetos (POO): herança simples, herança múltipla, polimorfismo, classes abstratas e encapsulamento, organizados em módulos.

O enunciado coloca a equipa no papel de uma empresa contratada para criar uma aplicação capaz de gerir um hotel, nomeadamente os seus **quartos** (de diferentes tipos, como simples e luxo), os **hóspedes** que os ocupam e os **funcionários** que trabalham no estabelecimento. Para além de funcionar, a solução tinha de seguir um modelo de classes definido pelo professor, com uma classe abstrata `Pessoa` no topo da hierarquia de pessoas e uma classe abstrata `Quarto` no topo da hierarquia de quartos.

A escolha do tema não é inocente: um hotel é um domínio onde as relações entre "coisas" do mundo real se traduzem de forma natural em classes. Um gerente *é um* funcionário; um funcionário *é uma* pessoa; um hóspede *tem um* quarto. Estas relações do tipo "é um" e "tem um" são precisamente a base da herança e da associação entre objetos, e permitem perceber, com exemplos concretos, para que servem os mecanismos da POO.

Objetivos do relatório

Este documento pretende, para além de descrever o produto final, **explicar o código na íntegra**. Assim, para cada módulo é apresentado o código fonte, é explicado o que faz cada classe, cada atributo e cada método, e são identificados, com exemplos, os locais onde foi aplicado cada conceito pedido no enunciado. Sempre que possível, as explicações são acompanhadas de resultados reais de execução do programa.

Estrutura do relatório

O relatório segue o modelo fornecido pela escola, adaptado a um projeto de programação, e encontra-se organizado da seguinte forma:

- **Capítulo I – Enquadramento e Caracterização do Projeto:** apresenta o enunciado, os requisitos, a equipa, as ferramentas e os critérios de avaliação.
- **Capítulo II – Atividades Desenvolvidas e Justificação Técnica:** é o capítulo central. Explica a arquitetura, cada módulo e cada classe, o programa principal, os conceitos de POO aplicados (com exemplos), os testes realizados e as dificuldades encontradas.
- **Capítulo III – Análise do Percurso Pessoal:** aprendizagens técnicas obtidas com o projeto.
- **Capítulo IV – Conclusão:** síntese dos resultados obtidos.

CAPÍTULO I — ENQUADRAMENTO E CARACTERIZAÇÃO DO PROJETO

1.1. Enquadramento

O projeto foi proposto como **1.º Trabalho Avaliativo** do módulo de programação em Python, lecionado pelo Professor Breno Sousa, a alunos do Curso Técnico de Programação e Gestão de Sistemas Informáticos (GPSI) da Escola Profissional Bento de Jesus Caraça. O trabalho foi realizado em equipa de três elementos e tinha uma duração prevista de **8 horas** de desenvolvimento.

Ao contrário do que acontece num estágio ou numa Formação em Contexto de Trabalho, em que o primeiro capítulo caracteriza uma empresa, neste relatório o primeiro capítulo caracteriza o **"cliente" fictício e o problema** que lhe foi apresentado: um hotel que precisa de um sistema informático simples, executado na consola, para gerir quartos, hóspedes e funcionários.

1.2. O problema proposto

Segundo o enunciado, o hotel necessita de um sistema para gerir, em resumo, **quartos de diferentes tipos** (simples e luxo) e **reservas feitas por hóspedes**. A aplicação devia ser escrita em Python e obedecer às seguintes restrições técnicas:

- uso de **herança simples**;
- uso de **herança múltipla**;
- uso de **polimorfismo**;
- uso de **classes abstratas**;
- uso obrigatório de **módulos** (o código dividido por ficheiros);
- uso de **estruturas de dados** (listas, dicionários, tuplos, etc.) sempre que necessário;
- uso de **laços de repetição** (`for`, `while`, etc.) sempre que necessário.

O enunciado inclui ainda dois diagramas com o modelo de herança que a equipa devia respeitar, bem como uma sugestão de métodos para cada classe, com a indicação de que seria permitido (e até desejável) acrescentar novas funções ou classes sempre que necessário.

Modelo de classes de pessoas

O primeiro diagrama do enunciado define que `Pessoa` é uma classe abstrata, da qual herdam diretamente `Hospede` e `Funcionario`. Por sua vez, `Funcionario` tem três subclasses: `Gerente`, `Recepcionista` e `TecnicoManutencao`. Por fim, `TecnicoRecepcao` herda em simultâneo de `Recepcionista` e de `TecnicoManutencao`, o que constitui o exemplo pedido de **herança múltipla**. A Figura 1 reproduz este modelo.

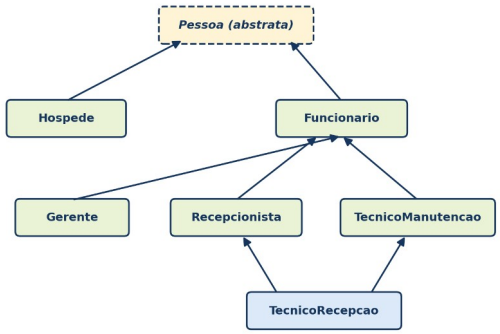


Figura 1 – Hierarquia de classes de pessoas (classes a tracejado são abstratas)

Modelo de classes de quartos

O segundo diagrama define `Quarto` como classe abstrata, com duas subclasses concretas: `QuartoSimples` e `QuartoLuxo`, como se mostra na Figura 2.

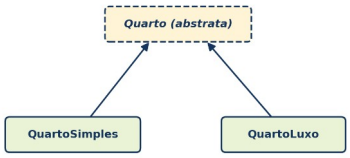


Figura 2 – Hierarquia de classes de quartos

1.3. Requisitos do sistema

A partir do enunciado, e das funcionalidades pedidas aos métodos de cada classe, a equipa definiu os seguintes requisitos.

Requisitos funcionais

Tabela 1 – Requisitos funcionais do sistema

Ref.	Requisito	Onde é satisfeito
RF1	Criar quartos de tipo simples ou luxo, com número e preço por dia, impedindo números repetidos e preços negativos.	`criar_quarto()` em main.py; classes `QuartoSimples` e `QuartoLuxo`
RF2	Listar todos os quartos, indicando tipo, preço e se estão ocupados.	`listar_quartos()`; `mostrar_informacoes()` de cada quarto
RF3	Registar hóspedes, associando-os a um quarto livre.	`criar_hospede()`; classe `Hospede`
RF4	Listar hóspedes e o valor total da conta de cada um.	`listar_hospedes()`; `Hospede.calcular_conta()`
RF5	Efetuar o checkout de um hóspede, libertando o quarto.	`fazer_checkout()`; `Hospede.checkout()`
RF6	Criar funcionários de quatro tipos (gerente, recepcionista, técnico de manutenção e técnico de receção).	`criar_funcionario()`; módulos `funcionario.py` e `tecnico_recepcao.py`
RF7	Imprimir um relatório com os dados dos funcionários.	`relatorio()`; `Gerente.gerar_relatorio()`

Ref.	Requisito	Onde é satisfeito
RF8	Registar reparos, escolhendo o técnico que os realiza.	`registrar_reparo()`; método `registar_reparo()` dos técnicos

Requisitos não funcionais e restrições

- A aplicação é executada na **consola** (interface de texto) e não utiliza bibliotecas externas, apenas o módulo `abc` da biblioteca padrão do Python.
- O código deve estar dividido em **módulos**, um por conceito (pessoa, quarto, hóspede, funcionários, técnico de receção) e um programa principal.
- Os dados são guardados **em memória** (listas); não existe persistência em ficheiro ou base de dados, uma vez que não foi pedida.
- O programa deve **validar** os dados introduzidos pelo utilizador, evitando valores impossíveis (idades negativas, preços negativos, números de quarto repetidos, etc.).

1.4. Ferramentas e recursos utilizados

Para o desenvolvimento do projeto e do relatório foram utilizadas as seguintes ferramentas:

- **Python 3:** linguagem do projeto, com recurso ao módulo `abc` (Abstract Base Classes) da biblioteca padrão, que é a única importação externa ao código escrito pela equipa.
- **Visual Studio Code:** editor de código usado para escrever e executar o programa.
- **Extensão Live Share (VS Code):** permitiu que os elementos da equipa editassem o mesmo código em simultâneo, cada um a partir do seu computador.
- **GitHub:** plataforma onde o trabalho foi guardado e partilhado entre os elementos da equipa.
- **Modelo de relatório:** documento fornecido pela escola, que serviu de base para este relatório.

1.5. Critérios de avaliação

O enunciado estabelece que a solução deve ser acompanhada por **um relatório do desenvolvimento** (o presente documento) e que cada aluno é avaliado individualmente. A classificação segue as fórmulas apresentadas na Tabela 2.

Tabela 2 – Fórmulas de classificação definidas no enunciado

Situação	Fórmula
Nota base	Nota = (Nota do Projeto × 0,85) + (Nota de Participação × 0,15)
Bónus por comprometimento e assiduidade (até 10%)	Nota Final = Nota + (Nota × 0,1)
Penalização por não participação (até 50%)	Nota Final = Nota − (Nota × 0,5)

O enunciado alerta ainda que, se algum elemento não participar no projeto, o professor deve ser informado; se não houver comunicação, todos os membros podem ser

penalizados, mas se a comunicação ocorrer antes de metade do tempo previsto, apenas o aluno que não estiver a participar é penalizado. Esta regra reforçou a importância de a equipa comunicar com regularidade durante as 8 horas disponíveis.

CAPÍTULO II — ATIVIDADES DESENVOLVIDAS E JUSTIFICAÇÃO TÉCNICA

Este capítulo explica, com o máximo de pormenor, tudo o que foi desenvolvido. Começa pela arquitetura geral do sistema, passa depois por cada módulo (ficheiro) e respetivas classes, explica o programa principal que une tudo e, por fim, identifica e exemplifica cada conceito de programação orientada a objetos aplicado. Os resultados de execução apresentados nas figuras foram obtidos ao correr o programa real, e não são simulações.

2.1. Arquitetura geral da solução

O sistema está dividido em **seis ficheiros Python** (módulos). Cinco deles definem classes e um (o `main.py`) contém o programa principal, isto é, o menu e as funções que interagem com o utilizador. Esta separação segue o princípio da **separação de responsabilidades**: as classes sabem como se comportam os objetos do hotel, enquanto o programa principal apenas pergunta dados ao utilizador, valida-os e pede às classes que façam o trabalho.

Tabela 3 – Módulos do projeto e respetiva responsabilidade

Módulo	Conteúdo	Depende de
`pessoa.py`	Classe abstrata `Pessoa`, base de hóspedes e funcionários	`abc`
`quarto.py`	Classe abstrata `Quarto` e subclasses `QuartoSimples` e `QuartoLuxo`	`abc`
`hospede.py`	Classe `Hospede`	`pessoa.py`
`funcionario.py`	`Funcionario`, `Gerente`, `Recepcionista` e `TecnicoManutencao`	`pessoa.py`
`tecnico_recepcao.py`	Classe `TecnicoRecepcao` (herança múltipla)	`funcionario.py`
`main.py`	Menu, validação de dados e listas globais	todos os anteriores

A Figura 3 mostra as dependências entre os módulos. As setas indicam quem importa quem: o `main.py` importa os restantes módulos; `hospede.py` e `funcionario.py` importam `pessoa.py`; e `tecnico_recepcao.py` importa as classes de `funcionario.py` de que precisa para a herança múltipla. Note-se que não existem dependências circulares, o que é um sinal de boa organização.

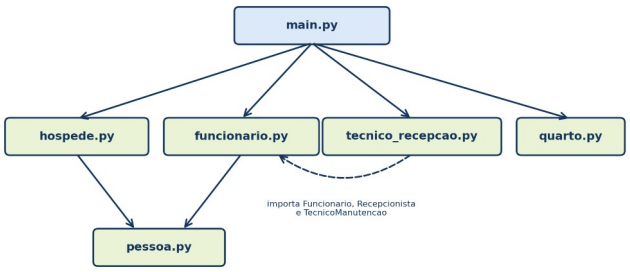


Figura 3 – Dependências (imports) entre os módulos do projeto

Uma decisão importante de desenho foi a de que os objetos **não conversam diretamente com o utilizador para pedir dados**: os métodos das classes apenas mostram informação (`print`) ou alteram o seu estado, enquanto todo o `input()` está concentrado no `main.py`. Assim, as classes podiam ser testadas sem menu, como se faz mais à frente (secção 2.9).

2.2. Módulo pessoa.py – a classe abstrata Pessoa

A classe `Pessoa` é a raiz da hierarquia de pessoas. Representa o que é comum a qualquer pessoa do sistema (um nome e uma idade) e obriga todas as classes descendentes a saberem apresentar a sua informação. O código completo do módulo é o seguinte:

Excerto 1 – ficheiro pessoa.py (completo)

```
from abc import ABC, abstractmethod

class Pessoa(ABC):
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    @property
    def nome(self):
        return self._nome

    @nome.setter
    def nome(self, valor):
        self._nome = valor

    @property
    def idade(self):
        return self._idade

    @idade.setter
    def idade(self, valor):
        if valor < 0:
            raise ValueError("A idade não pode ser negativa!")
        self._idade = valor

    @abstractmethod
    def mostrar_informacoes(self):
        pass
```

2.2.1. Herdar de ABC: o que torna a classe abstrata

A linha `class Pessoa(ABC)` faz com que `Pessoa` herde de `ABC` (*Abstract Base Class*), importada do módulo `abc`. Uma classe que herda de `ABC` e tem pelo menos um método marcado com `@abstractmethod` passa a ser **abstrata**: o Python recusa-se a criar objetos diretamente a partir dela. Isto faz todo o sentido no domínio do problema, porque num hotel nunca existe "uma pessoa" em abstrato: existe sempre um hóspede ou um funcionário.

O método abstrato é `mostrar_informacoes()`. Está definido apenas com `pass`, ou seja, **sem implementação**: funciona como um "contrato" que diz às subclasses "tens de saber mostrar a tua informação, mas a forma de o fazer é contigo". Se uma subclasse não o implementar, também ela continua abstrata e não pode ser instanciada. A Figura 9 (secção 2.8) mostra o erro que o Python devolve ao tentar criar uma `Pessoa`.

2.2.2. O construtor __init__

O construtor `__init__(self, nome, idade)` recebe os dois dados fundamentais e guarda-os. Repare-se que as linhas `self.nome = nome` e `self.idade = idade` **não escrevem diretamente nos atributos**: como `nome` e `idade` foram definidos como propriedades (`property`), estas atribuições chamam automaticamente os respetivos *setters*, o que significa que a validação da idade acontece logo na criação do objeto.

2.2.3. Encapsulamento com property e setter

Os verdadeiros valores ficam guardados em atributos "protegidos", `_nome` e `_idade`, cujo sublinhado inicial é a convenção do Python para indicar que não devem ser acedidos de fora da classe. O acesso do exterior é feito através de duas propriedades:

- `@property def nome(self)` – **getter**: permite escrever `pessoa.nome` como se fosse um atributo normal, mas executa o método por baixo;
- `@nome.setter` – **setter**: é chamado quando se escreve `pessoa.nome = "Ana"`. No caso do nome limita-se a guardar o valor;
- `@idade.setter` – setter da idade com **validação**: se o valor for negativo, lança `ValueError("A idade não pode ser negativa!")`, impedindo que o objeto fique num estado inválido.

Exemplo de encapsulamento
Em `pessoa.py`, o setter da idade rejeita valores negativos. Ao executar `h.idade = -3` numa instância de `Hospede`, o programa devolve `ValueError -> A idade não pode ser negativa!` (resultado real na Figura 10). A regra fica escrita **uma única vez**, em `Pessoa`, e é herdada por todas as subclasses.

Esta abordagem cumpre o que o enunciado pede à classe `Pessoa`: `__init__`, `@property nome`, `@nome.setter`, `@property idade`, `@idade.setter` que impede valores negativos e `mostrar_informacoes` como método abstrato.

2.3. Módulo quarto.py – Quarto, QuartoSimples e QuartoLuxo

Este módulo reúne a classe abstrata `Quarto` e as suas duas subclasses. A decisão de as colocar no mesmo ficheiro prende-se com o facto de serem muito pequenas e de formarem um conjunto coeso.

Excerto 2 – ficheiro quarto.py (completo)

```
from abc import ABC, abstractmethod

class Quarto(ABC):
    def __init__(self, numero, tipo, preco_diaria):
        self.numero = numero
        self.tipo = tipo
        self.preco_diaria = preco_diaria
        self._ocupado = False

    @property
    def ocupado(self):
        return self._ocupado

    @ocupado.setter
    def ocupado(self, valor):
        self._ocupado = valor
```

```
    @property
    def preco_diaria(self):
        return self._preco_diaria

    @preco_diaria.setter
    def preco_diaria(self, valor):
        if valor < 0:
            raise ValueError("Preço da diária não pode ser negativo!")
        self._preco_diaria = valor

    @abstractmethod
    def mostrar_informacoes(self):
        pass


class QuartoSimples(Quarto):
    def __init__(self, numero, preco_diaria):
        super().__init__(numero, "Simples", preco_diaria)

    def mostrar_informacoes(self):
        status = "Sim" if self.ocupado else "Não"
        print(f"Quarto: {self.numero} | Tipo: {self.tipo} | Preço/Dia: {self.preco_diaria}€ |
Ocupado: {status}")


class QuartoLuxo(Quarto):
    def __init__(self, numero, preco_diaria):
        super().__init__(numero, "Luxo", preco_diaria)

    def mostrar_informacoes(self):
        status = "Sim" if self.ocupado else "Não"
        print(f"Quarto: {self.numero} | Tipo: {self.tipo} | Preço/Dia: {self.preco_diaria}€ |
Ocupado: {status}")
```

2.3.1. A classe abstrata Quarto

`Quarto` guarda o que é comum a todos os quartos: o `numero`, o `tipo` (texto), o `preco_diaria` e o estado `_ocupado`, que começa sempre em `False` (todo o quarto novo está livre). Tal como em `Pessoa`, existe um método abstrato, `mostrar_informacoes()`, e portanto não é possível criar um `Quarto` "genérico".

Duas propriedades protegem o estado do objeto:

- `ocupado` tem getter e setter simples. É o `Hospede` que, ao ser associado ao quarto, o coloca a `True`, e é o checkout que o repõe a `False` (ver secção 2.4).
- `preco_diaria` tem um setter com validação: se o valor for negativo é lançada uma `ValueError`. Como o construtor faz `self.preco_diaria = preco_diaria`, a validação ocorre também na criação. Isto corresponde ao requisito "@preco_diaria.setter: impede preços negativos".

2.3.2. As subclasses QuartoSimples e QuartoLuxo

Cada subclasse chama o construtor da classe-mãe com `super().__init__(numero, "Simples", preco_diaria)` (ou `"Luxo"`), fixando o tipo e dispensando o utilizador de o indicar. Esta é uma utilização típica de **herança simples**: as subclasses reaproveitam toda a lógica de `Quarto` (propriedades, validação, estado) e limitam-se a especializar o necessário.

Ambas implementam `mostrar_informacoes()` (obrigatório por serem concretas) e usam uma expressão condicional para converter o booleano em texto: `status =`

`"Sim" if self.ocupado else "Não"`. Embora hoje o texto produzido seja igual nas duas, a existência de um método próprio em cada classe permite que, no futuro, o quarto de luxo mostre informação adicional (como comodidades) sem alterar o restante código.

Exemplo de herança simples
A classe `QuartoLuxo(Quarto)` herda de uma única classe-mãe. Ao fazer `QuartoLuxo(7, 100)`, o objeto já possui `ocupado`, `preco_diaria` e as validações, sem que `QuartoLuxo` as tenha escrito. Se `Quarto` mudar, todos os tipos de quarto beneficiam da mudança.

2.4. Módulo hospede.py – a classe Hospede

A classe `Hospede` herda de `Pessoa` e representa quem fica alojado no hotel. Para além do nome e da idade, tem os dias de estadia e (opcionalmente) um quarto associado.

Excerto 3 – ficheiro hospede.py (completo)

```
from pessoa import Pessoa

class Hospede(Pessoa):
    def __init__(self, nome, idade, dias_estadia, quarto=None):
        super().__init__(nome, idade)
        self.dias_estadia = dias_estadia
        self._quarto = None

        if quarto is not None:
            self.atribuir_quarto(quarto)

    @property
    def quarto(self):
        return self._quarto

    @quarto.setter
    def quarto(self, quarto):
        if quarto is not None:
            if not quarto.ocupado:
                self._quarto = quarto
                quarto.ocupado = True
            else:
                print("Aviso: O quarto selecionado já se encontra ocupado!")

    def atribuir_quarto(self, quarto):
        self.quarto = quarto

    def calcular_conta(self):
        if self.quarto:
            return self.dias_estadia * self.quarto.preco_diaria
        return 0

    def mostrar_informacoes(self):
        num_quarto = self.quarto.numero if self.quarto else "Nenhum"
        print(f"Hóspede: {self.nome} | Idade: {self.idade} | Dias de Estadia: {self.dias_estadia} |
Quarto: {num_quarto}")

    def checkout(self):
        if self.quarto:
            self.quarto.ocupado = False
            self._quarto = None
            print("Checkout realizado com sucesso e quarto libertado!")
```

2.4.1. Construtor e herança

O construtor recebe `nome`, `idade`, `dias_estadia` e `quarto=None`. A primeira instrução, `super().__init__(nome, idade)`, delega na classe `Pessoa` a inicialização e validação dos dados comuns. Depois, `self._quarto = None` inicializa o atributo protegido do quarto, e só então, se tiver sido fornecido um quarto, é chamado `self.atribuir_quarto(quarto)`. Esta ordem é relevante: o atributo `_quarto` tem de existir **antes** de se chamar o setter, caso contrário o `self.quarto` poderia falhar.

2.4.2. A propriedade quarto e a regra de ocupação

O setter da propriedade `quarto` implementa a regra de negócio mais importante do hotel: **um quarto só pode ser atribuído se estiver livre**. O fluxo do setter é o seguinte:

1. Se o quarto recebido não for `None`, verifica `quarto.ocupado`.
2. Se estiver livre, guarda a referência em `self._quarto` e marca o quarto como ocupado (`quarto.ocupado = True`).
3. Se estiver ocupado, escreve um aviso e **não altera** o hóspede.

Este comportamento é uma **associação** entre dois objetos: o hóspede guarda uma referência para o objeto `Quarto` (relação "tem um"). A Figura 4 resume a relação. Como se trata de uma referência e não de uma cópia, quando o hóspede marca o quarto como ocupado o próprio objeto que está na lista `quartos` do `main.py` fica atualizado, o que explica porque é que `listar_quartos()` mostra "Ocupado: Sim" sem ser preciso fazer mais nada.

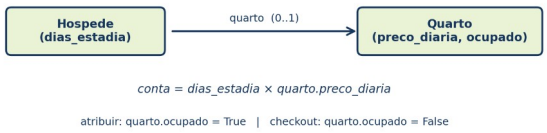


Figura 4 – Associação entre Hospede e Quarto

2.4.3. Cálculo da conta, informação e checkout

- `atribuir_quarto(quarto)` limita-se a fazer `self.quarto = quarto`, isto é, reutiliza o setter. Existe por ser um dos métodos sugeridos no enunciado.
- `calcular_conta()` devolve `dias_estadia * quarto.preco_diaria` se o hóspede tiver quarto, ou `0` caso contrário. Por exemplo, 3 dias num quarto de 45 € custam 135 €.
- `mostrar_informacoes()` implementa o método abstrato de `Pessoa`. Usa `self.quarto.numero if self.quarto else "Nenhum"` para evitar um erro se não houver quarto.
- `checkout()` liberta o quarto (`ocupado = False`), remove a associação (`_quarto = None`) e imprime uma mensagem.

Na Figura 11 (secção 2.9) é possível ver este comportamento em ação: depois de dois hóspedes serem registados, ambos os quartos aparecem como ocupados; depois do checkout de um deles, o respetivo quarto volta a aparecer como livre.

2.5. Módulo funcionario.py – a hierarquia de funcionários

O módulo `funcionario.py` contém quatro classes: a classe-base `Funcionario` e as suas três especializações, `Gerente`, `Recepcionista` e `TecnicoManutencao`. É aqui que a **herança simples** e o **polimorfismo** mais se notam, porque todas as classes redefinem o mesmo método, `mostrar_informacoes()`, cada uma à sua maneira.

2.5.1. Funcionario

Excerto 4 – classe Funcionario

```
from pessoa import Pessoa

class Funcionario(Pessoa):
    def __init__(self, nome, idade, salario):
        super().__init__(nome, idade)
        self.salario = salario

    def mostrar_informacoes(self):
        print(f"Funcionário: {self.nome} | Idade: {self.idade} | Salário: {self.salario}€")
```

`Funcionario` herda de `Pessoa` e acrescenta o **salário**. O construtor chama `super().__init__(nome, idade)` para reaproveitar a inicialização de `Pessoa` e só depois guarda o salário. Ao contrário do que acontece com `Pessoa`, esta classe **implementa** `mostrar_informacoes()`, o que a torna uma classe concreta, isto é, é possível criar um `Funcionario` "genérico". Esta decisão tem uma vantagem: as subclasses que não quiserem personalizar a apresentação podem reutilizar o método herdado.

2.5.2. Gerente

Excerto 5 – classe Gerente

```
class Gerente(Funcionario):
    def __init__(self, nome, idade, salario, bonus):
        super().__init__(nome, idade, salario)
        self.bonus = bonus

    def mostrar_informacoes(self):
        total = self.salario + self.bonus
        print(f"Gerente: {self.nome} | Idade: {self.idade} | Salário Base: {self.salario}€ | Bónus:
{self.bonus}€ | Total: {total}€")

    def gerar_relatorio(self, funcionarios):
        print("\n--- RELATÓRIO DE FUNCIONÁRIOS (GERADO PELO GERENTE) ---")
        for f in funcionarios:
            f.mostrar_informacoes()
```

O `Gerente` acrescenta o atributo `bonus`. O seu `mostrar_informacoes()` **sobrescreve** (*override*) o da classe-mãe para mostrar o salário base, o bónus e o total, calculado na hora (`total = self.salario + self.bonus`). Cumpre-se assim o requisito "mostra salário + bónus".

O método mais interessante é `gerar_relatorio(self, funcionarios)`. Recebe uma lista com qualquer tipo de funcionário e percorre-a com um `for`, chamando `f.mostrar_informacoes()` em cada elemento. O gerente não precisa de saber de que tipo é cada funcionário: cada objeto responde com a sua própria versão do método.

Este é o exemplo mais claro de **polimorfismo** no projeto, que é aprofundado na secção 2.8.

2.5.3. Recepcionista

Excerto 6 – classe Recepcionista

```
class Recepcionista(Funcionario):
    def __init__(self, nome, id_func, idade, salario, turno):
        super().__init__(nome, idade, salario)
        self.id_func = id_func
        self.turno = turno

    @property
    def turno(self):
        return self._turno

    @turno.setter
    def turno(self, turno):
        if turno.lower() in ["manhã", "tarde", "noite"]:
            self._turno = turno.lower()
        else:
            print("Turno inválido! Mantenha: manhã, tarde ou noite.")
            self._turno = "manhã"

    def mostrar_informacoes(self):
        print(f"Recepcionista: {self.nome} (ID: {self.id_func}) | Idade: {self.idade} | Turno:
{self.turno} | Salário: {self.salario}€")

    def registar_hospede(self, hospede, lista_hospedes):
        lista_hospedes.append(hospede)
        print(f"Hóspede {hospede.nome} registado com sucesso!")

    def listar_hospedes(self, lista_hospedes):
        print("\n--- LISTA DE HÓSPEDES ---")
        for hospede in lista_hospedes:
            hospede.mostrar_informacoes()
```

A `Recepcionista` acrescenta dois atributos: o `id_func` (identificador único) e o `turno`. O turno é uma propriedade com **regra de validação**: o setter só aceita "manhã", "tarde" ou "noite" (em qualquer combinação de maiúsculas e minúsculas, graças a `turno.lower()`), guardando o valor sempre em minúsculas. Se for introduzido outro valor, é escrito um aviso e o turno fica "manhã" por omissão. Isto garante que um recepcionista nunca fica com um turno inexistente.

Os métodos próprios desta classe são:

- `registar_hospede(hospede, lista_hospedes)` – acrescenta o hóspede à lista recebida com `append` e confirma o registo;
- `listar_hospedes(lista_hospedes)` – percorre a lista com um `for` e pede a cada hóspede que se mostre (`hospede.mostrar_informacoes()`);
- `mostrar_informacoes()` – sobrescrito, inclui o ID e o turno.

Note-se que o enunciado sugere os nomes `registrar_hospede` e `listar_hospedes`. A equipa optou pela grafia de Portugal (`registar`), mantendo a mesma função.

2.5.4. TecnicoManutencao

Excerto 7 – classe TecnicoManutencao

```
class TecnicoManutencao(Funcionario):
```

```
    def __init__(self, nome, idade, salario, especialidade):
        super().__init__(nome, idade, salario)
        self.especialidade = especialidade

    @property
    def especialidade(self):
        return self._especialidade

    @especialidade.setter
    def especialidade(self, especialidade):
        self._especialidade = especialidade

    def mostrar_informacoes(self):
        print(f"Técnico de Manutenção: {self.nome} | Idade: {self.idade} | Especialidade:
{self.especialidade} | Salário: {self.salario}€")

    def registar_reparo(self, descricao):
        print(f"Reparo registado por {self.nome}: {descricao}")
```

O `TecnicoManutencao` define a área técnica (`especialidade`), também como propriedade com getter e setter, tal como pedido no enunciado. O método `registar_reparo(descricao)` escreve a mensagem "Reparo registado por …". Este método é importante porque é **redefinido** pela classe `TecnicoRecepcao` (secção seguinte), constituindo um segundo exemplo de polimorfismo.

2.6. Módulo tecnico_recepcao.py – herança múltipla

A classe `TecnicoRecepcao` representa um funcionário que acumula duas funções: a de recepcionista e a de técnico de manutenção. No modelo do enunciado, é a única classe com **duas classes-mãe**, e portanto o exemplo de herança múltipla.

Excerto 8 – ficheiro tecnico_recepcao.py (completo)

```
from funcionario import Funcionario, Recepcionista, TecnicoManutencao

class TecnicoRecepcao(Recepcionista, TecnicoManutencao):
    def __init__(self, nome, id_func, idade, salario, setor, especialidade):
        Funcionario.__init__(self, nome, idade, salario)
        self.id_func = id_func
        self.turno = "manhã"
        self.especialidade = especialidade
        self.setor = setor

    def registar_reparo(self, descricao):
        print(f"Reparo registado na Receção (Setor {self.setor}) por {self.nome}: {descricao}")

    def mostrar_informacoes(self):
        print(f"Técnico de Receção: {self.nome} (ID: {self.id_func}) | Setor: {self.setor} |
Especialidade: {self.especialidade} | Salário: {self.salario}€")
```

2.6.1. O que herda e o que acrescenta

Ao declarar `class TecnicoRecepcao(Recepcionista, TecnicoManutencao)`, a classe passa a possuir, sem escrever uma linha adicional, tudo o que existe em ambas as classes-mãe: de `Recepcionista` herda o `id_func`, o `turno`, o `registar_hospede` e o `listar_hospedes`; de `TecnicoManutencao` herda a `especialidade`. Acrescenta apenas o atributo `setor` e **redefine** dois métodos:

- `registar_reparo(descricao)` – em vez da mensagem genérica, indica que o reparo foi registado na Receção, no setor correspondente;
- `mostrar_informacoes()` – apresenta, num só método, os dados das duas funções (ID, setor, especialidade e salário), como pede o enunciado ("exibe dados de ambas as funções").

2.6.2. Ordem de resolução de métodos (MRO)

Quando uma classe tem várias classes-mãe, o Python precisa de uma regra para decidir onde procurar um método. Essa regra chama-se **MRO** (*Method Resolution Order*) e é calculada com o algoritmo C3. Para `TecnicoRecepcao`, a ordem é a da Figura 5: primeiro a própria classe, depois `Recepcionista`, depois `TecnicoManutencao`, e só então as classes comuns `Funcionario`, `Pessoa`, `ABC` e `object`.



Figura 5 – Ordem de resolução de métodos (MRO) da classe TecnicoRecepcao

Esta hierarquia tem a forma de um "diamante": `Funcionario` é ancestral de `Recepcionista` e de `TecnicoManutencao`, que por sua vez são ambas ancestrais de `TecnicoRecepcao`. É o chamado **problema do diamante**, e o Python resolve-o garantindo que `Funcionario` e `Pessoa` aparecem **uma só vez** na MRO, e que cada classe aparece antes das suas classes-mãe.

2.6.3. Um problema encontrado: super() e herança múltipla

Durante os testes da opção "Técnico de Receção", a equipa encontrou um erro interessante, que ilustra bem a subtileza da herança múltipla. A primeira versão do construtor chamava `Recepcionista.__init__(self, nome, id_func, idade, salario, "manhã")`. Dentro deste método, a instrução `super().__init__(nome, idade, salario)` **não** chama `Funcionario`, como seria de esperar quando se olha apenas para `Recepcionista`: chama a **próxima classe da MRO do objeto**, que, para um `TecnicoRecepcao`, é `TecnicoManutencao`. Como o construtor de `TecnicoManutencao` exige um quarto argumento (`especialidade`), o programa terminava com o erro:

```
TypeError: TecnicoManutencao.__init__() missing 1 required positional argument: 'especialidade'
```

O erro mostra que `super()` não significa "a minha classe-mãe", mas sim "a seguinte na MRO do objeto concreto". Para o corrigir, o construtor de `TecnicoRecepcao` passou a inicializar diretamente a classe-base comum, `Funcionario.__init__(self, nome, idade, salario)`, e a atribuir ele próprio os atributos específicos (`id_func`, `turno`, `especialidade` e `setor`), como se pode ver no Excerto 8. Depois desta alteração, a criação de técnicos de receção passou a funcionar sem erros (Figura 12).

Alternativa mais "pura"

A solução mais elegante em Python seria usar **herança cooperativa**: todos os construtores aceitariam `**kwargs` e chamariam `super().__init__(**kwargs)`. Não foi adotada por ser mais complexa do que o necessário para este trabalho, mas é a abordagem recomendada em projetos maiores.

2.7. Módulo main.py – o programa principal

O ficheiro `main.py` é o ponto de entrada da aplicação. Importa as classes necessárias e implementa um menu em consola que permite ao utilizador usar todas as funcionalidades. Nas subsecções seguintes reproduz-se e explica-se **todo o código** deste ficheiro, por partes.

2.7.1. Importações e estruturas de dados globais

Excerto 9 – início do main.py

```
from hospede import Hospede
from funcionario import Gerente, Recepcionista, TecnicoManutencao
from tecnico_recepcao import TecnicoRecepcao
from quarto import QuartoSimples, QuartoLuxo

quartos = []
hospedes = []
funcionarios = []
```

As cinco primeiras linhas importam as classes dos outros módulos (uso obrigatório de **módulos**). Em seguida são criadas três **listas** vazias, `quartos`, `hospedes` e `funcionarios`, que funcionam como a "base de dados" em memória do hotel. Foram escolhidas listas porque a ordem de inserção é útil na apresentação e porque o número de elementos é pequeno; para pesquisas por chave (por exemplo, encontrar um quarto pelo número) poderia ter-se usado um **dicionário**.

2.7.2. Funções de gestão de quartos

Excerto 10 – criar_quarto() e listar_quartos()

```
def criar_quarto():
    print("\n--- CRIAR QUARTO ---")
    numero_str = input("Número do quarto: ")
    if not numero_str.isdigit():
        print("Número inválido!")
        return
    numero = int(numero_str)

    for quarto in quartos:
        if quarto.numero == numero:
            print("Já existe um quarto com esse número!")
            return

    print("1 - Simples\n2 - Luxo")
    tipo = input("Escolha o tipo: ")

    preco = float(input("Preço por dia (€): "))
    if preco < 0:
        print("Preço não pode ser negativo!")
        return

    if tipo == "1":
        quarto = QuartoSimples(numero, preco)
    elif tipo == "2":
```

```
        quarto = QuartoLuxo(numero, preco)
    else:
        print("Tipo inválido!")
        return

    quartos.append(quarto)
    print("Quarto criado com sucesso!")


def listar_quartos():
    print("\n--- LISTA DE QUARTOS ---")
    if not quartos:
        print("Não existem quartos registados.")
        return

    for quarto in quartos:
        quarto.mostrar_informacoes()
```

`criar_quarto()` lê o número do quarto como texto e usa `isdigit()` para garantir que é um número inteiro positivo (a conversão com `int()` só é feita depois da validação, evitando o erro de converter texto). Seguidamente percorre a lista `quartos` com um `for` para impedir números repetidos. Pede o tipo (1 – Simples, 2 – Luxo) e o preço, rejeita preços negativos e, com um `if / elif / else`, cria o objeto da subclasse correta. Esta escolha da classe a instanciar em função da opção do utilizador é uma forma simples de **fábrica de objetos**.

`listar_quartos()` verifica primeiro se a lista está vazia (`if not quartos`) e, caso não esteja, percorre-a chamando `quarto.mostrar_informacoes()`. É aqui que o polimorfismo dos quartos é usado: a lista pode misturar `QuartoSimples` e `QuartoLuxo` e o código é o mesmo para ambos.

2.7.3. Funções de gestão de hóspedes

Excerto 11 – criar_hospede(), listar_hospedes() e fazer_checkout()

```
def criar_hospede():
    print("\n--- REGISTAR HÓSPEDE ---")
    nome = input("Nome: ")

    idade_str = input("Idade: ")
    if not idade_str.isdigit():
        print("Idade inválida!")
        return
    idade = int(idade_str)
    if idade <= 0:
        print("Idade inválida!")
        return

    dias_str = input("Dias de estadia: ")
    if not dias_str.isdigit():
        print("Dias de estadia inválidos!")
        return
    dias_estadia = int(dias_str)
    if dias_estadia <= 0:
        print("Dias de estadia inválidos!")
        return

    if not quartos:
        print("Não existem quartos disponíveis no hotel!")
        return

    print("\nQuartos Existentes:")
    for quarto in quartos:
```

```
        quarto.mostrar_informacoes()

    numero_str = input("\nNúmero do quarto para alojar: ")
    if not numero_str.isdigit():
        print("Número inválido!")
        return
    numero = int(numero_str)

    quarto_escolhido = None
    for quarto in quartos:
        if quarto.numero == numero:
            quarto_escolhido = quarto
            break

    if quarto_escolhido is None:
        print("Quarto não encontrado!")
        return

    if quarto_escolhido.ocupado:
        print("O quarto selecionado já está ocupado!")
        return

    hospede = Hospede(nome, idade, dias_estadia, quarto_escolhido)
    hospedes.append(hospede)
    print("Hóspede registado com sucesso!")


def listar_hospedes():
    print("\n--- LISTA DE HÓSPEDES ---")
    if not hospedes:
        print("Não existem hóspedes registados.")
        return

    for hospede in hospedes:
        hospede.mostrar_informacoes()
        print(f"Valor Total da Conta: {hospede.calcular_conta():.2f}€")
        print("-" * 35)


def fazer_checkout():
    print("\n--- CHECKOUT ---")
    nome = input("Nome do hóspede: ")

    hospede_encontrado = None
    for hospede in hospedes:
        if hospede.nome.lower() == nome.lower():
            hospede_encontrado = hospede
            break

    if hospede_encontrado is None:
        print("Hóspede não encontrado!")
        return

    print(f"Valor total a pagar: {hospede_encontrado.calcular_conta():.2f}€")
    hospede_encontrado.checkout()
    hospedes.remove(hospede_encontrado)
```

`criar_hospede()` é a função de validação mais extensa. Pede e valida a idade e os dias de estadia (inteiros positivos); verifica se existem quartos; mostra os quartos existentes; pede o número do quarto e procura-o com um `for`, usando `break` assim que o encontra; e rejeita o quarto se não existir ou se já estiver ocupado. Só depois de todas as verificações cria o `Hospede` e o acrescenta à lista. A verificação `quarto_escolhido.ocupado` é, de propósito, feita no `main.py` **e** no setter da classe

`Hospede`: o menu dá uma mensagem amigável logo à partida e a classe garante a regra mesmo que seja usada por outro programa (programação defensiva).

`listar_hospedes()` percorre a lista de hóspedes e, para cada um, chama `mostrar_informacoes()` e imprime o valor da conta com duas casas decimais (`:.2f`), seguido de uma linha de 35 hífenes (`"-" * 35`) como separador.

`fazer_checkout()` pede o nome, procura o hóspede ignorando maiúsculas e minúsculas (`.lower()` nos dois lados da comparação), mostra o valor a pagar, chama `hospede.checkout()` (que liberta o quarto) e remove o hóspede da lista com `remove`.

2.7.4. Criação de funcionários

Excerto 12 – criar_funcionario()

```
def criar_funcionario():
    print("\n--- CRIAR FUNCIONÁRIO ---")
    print("1 - Gerente\n2 - Recepcionista\n3 - Técnico de Manutenção\n4 - Técnico de Receção")

    opcao = input("Escolha uma opção: ")
    if opcao not in ["1", "2", "3", "4"]:
        print("Opção inválida!")
        return

    nome = input("Nome: ")

    idade_str = input("Idade: ")
    if not idade_str.isdigit():
        print("Idade inválida!")
        return
    idade = int(idade_str)
    if idade <= 0:
        print("Idade inválida!")
        return

    salario = float(input("Salário (€): "))
    if salario <= 0:
        print("Salário inválido!")
        return

    if opcao == "1":
        bonus = float(input("Bónus (€): "))
        if bonus < 0:
            print("Bónus inválido!")
            return
        funcionario = Gerente(nome, idade, salario, bonus)

    elif opcao == "2":
        id_str = input("ID do funcionário: ")
        if not id_str.isdigit():
            print("ID inválido!")
            return
        id_func = int(id_str)
        if id_func <= 0:
            print("ID inválido!")
            return

        for f in funcionarios:
            if hasattr(f, 'id_func') and f.id_func == id_func:
                print("Já existe um funcionário com esse ID!")
                return

        turno = input("Turno (manhã/tarde/noite): ")
        funcionario = Recepcionista(nome, id_func, idade, salario, turno)
```

```
    elif opcao == "3":
        especialidade = input("Especialidade: ")
        funcionario = TecnicoManutencao(nome, idade, salario, especialidade)

    elif opcao == "4":
        id_str = input("ID do funcionário: ")
        if not id_str.isdigit():
            print("ID inválido!")
            return
        id_func = int(id_str)
        if id_func <= 0:
            print("ID inválido!")
            return

        for f in funcionarios:
            if hasattr(f, 'id_func') and f.id_func == id_func:
                print("Já existe um funcionário com esse ID!")
                return

        setor = input("Setor: ")
        especialidade = input("Especialidade: ")
        funcionario = TecnicoRecepcao(nome, id_func, idade, salario, setor, especialidade)

    funcionarios.append(funcionario)
    print("Funcionário criado com sucesso!")
```

`criar_funcionario()` começa por mostrar as quatro categorias e validar a opção. De seguida, pede os dados comuns (nome, idade, salário) e, em função da opção, pede os dados específicos: o bónus para o `Gerente`; o ID e o turno para o `Recepcionista`; a especialidade para o `TecnicoManutencao`; e o ID, o setor e a especialidade para o `TecnicoRecepcao`. Para os funcionários com `id_func`, a função percorre a lista `funcionarios` com um `for` e usa `hasattr(f, "id_func")` para só comparar IDs quando o atributo existe (um `Gerente`, por exemplo, não tem ID). Isto impede dois funcionários com o mesmo ID.

2.7.5. Relatório e registo de reparos

Excerto 13 – relatorio() e registrar_reparo()

```
def relatorio():
    print("\n--- RELATÓRIO DO HOTEL ---")
    if not funcionarios:
        print("Não existem funcionários registados.")
        return

    gerente = next((f for f in funcionarios if isinstance(f, Gerente)), None)
    if gerente:
        gerente.gerar_relatorio(funcionarios)
    else:
        for f in funcionarios:
            f.mostrar_informacoes()


def registrar_reparo():
    print("\n--- REGISTAR REPARO ---")
    descricao = input("Descrição do reparo: ")
    tecnicos = [f for f in funcionarios if isinstance(f, TecnicoManutencao)]

    if not tecnicos:
        print("Não existem técnicos registados no sistema!")
        return
```

```
    for i, tecnico in enumerate(tecnicos):
        print(f"{i + 1} - {tecnico.nome} ({tecnico.__class__.__name__})")

    escolha_str = input("Escolha o técnico: ")
    if not escolha_str.isdigit():
        print("Opção inválida!")
        return
    escolha = int(escolha_str)

    if escolha < 1 or escolha > len(tecnicos):
        print("Opção inválida!")
        return

    tecnicos[escolha - 1].registar_reparo(descricao)
```

`relatorio()` procura um gerente na lista usando uma expressão geradora com `next`: `next((f for f in funcionarios if isinstance(f, Gerente)), None)`. Se encontrar um gerente, é este que gera o relatório (`gerente.gerar_relatorio(funcionarios)`), o que reflete a regra do negócio de que é o gerente quem emite o relatório. Se não existir gerente, o programa mostra a informação de cada funcionário diretamente, em vez de falhar.

`registrar_reparo()` constrói, com uma **compreensão de lista**, a lista de todos os funcionários que são `TecnicoManutencao`: `[f for f in funcionarios if isinstance(f, TecnicoManutencao)]`. Como `TecnicoRecepcao` herda de `TecnicoManutencao`, o `isinstance` devolve `True` para ele também, e o técnico de receção aparece na lista de escolha, mesmo não sendo um "técnico de manutenção" em sentido estrito (Figura 13). Esta é uma vantagem direta da herança: um objeto de uma subclasse pode ser usado onde se espera um objeto da superclasse. A função usa `enumerate` para numerar os técnicos a partir de 1 e valida a escolha antes de chamar `registar_reparo(descricao)` no técnico escolhido.

2.7.6. O menu principal

Excerto 14 – ciclo do menu principal

```
while True:
    print("\n===== GESTÃO DO HOTEL =====")
    print("1 - Criar quarto")
    print("2 - Listar quartos")
    print("3 - Registar hóspede")
    print("4 - Listar hóspedes")
    print("5 - Fazer checkout")
    print("6 - Criar funcionário")
    print("7 - Imprimir relatório")
    print("8 - Registar reparo")
    print("0 - Sair")

    opcao = input("Escolha uma opção: ")

    if opcao == "1":
        criar_quarto()
    elif opcao == "2":
        listar_quartos()
    elif opcao == "3":
        criar_hospede()
    elif opcao == "4":
        listar_hospedes()
    elif opcao == "5":
        fazer_checkout()
    elif opcao == "6":
```

```
        criar_funcionario()
    elif opcao == "7":
        relatorio()
    elif opcao == "8":
        registrar_reparo()
    elif opcao == "0":
        print("A encerrar o sistema do hotel... Até breve!")
        break
    else:
        print("Opção inválida! Tente novamente.")
```

O menu é um ciclo `while True` que mostra as opções, lê a escolha e chama a função correspondente através de uma cadeia `if / elif`. A opção `0` executa `break`, que termina o ciclo e o programa. Qualquer outra opção resulta numa mensagem de erro e o menu é mostrado de novo. O fluxograma da Figura 6 resume este funcionamento.

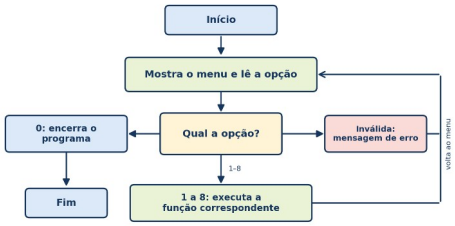


Figura 6 – Fluxograma do menu principal

2.8. Conceitos de POO aplicados, com exemplos

Esta secção responde diretamente ao que é pedido no enunciado: identifica **onde** foi usado cada conceito, **o que** faz e **porquê** foi usado. Cada conceito inclui pelo menos um exemplo concreto retirado do código.

2.8.1. Classes, objetos, atributos e métodos

Uma **classe** é o molde que define os atributos (dados) e os métodos (comportamentos) de um tipo de coisa; um **objeto** (ou instância) é uma coisa concreta criada a partir desse molde. No projeto existem 10 classes: `Pessoa`, `Hospede`, `Funcionario`, `Gerente`, `Recepcionista`, `TecnicoManutencao`, `TecnicoRecepcao`, `Quarto`, `QuartoSimples` e `QuartoLuxo`.

Exemplo
A instrução `quarto = QuartoSimples(numero, preco)`, em `criar_quarto()`, cria um **objeto** da classe `QuartoSimples`. O método especial `__init__` (construtor) é chamado automaticamente e define os atributos `numero`, `tipo`, `preco_diaria` e `_ocupado`. O método `quarto.mostrar_informacoes()` é um **método de instância**: `self` representa o objeto sobre o qual é chamado.

2.8.2. Herança simples

Na herança simples, uma classe herda de **uma única** classe-mãe, reaproveitando os seus atributos e métodos e podendo acrescentar ou alterar comportamento. Foi usada em todas as relações do modelo, exceto em `TecnicoRecepcao`.

Tabela 4 – Relações de herança simples no projeto

Classe-filha	Classe-mãe	O que acrescenta
`Hospede`	`Pessoa`	`dias_estadia` e associação a um `Quarto`
`Funcionario`	`Pessoa`	`salario`
`Gerente`	`Funcionario`	`bonus` e `gerar_relatorio()`
`Recepcionista`	`Funcionario`	`id_func`, `turno`, registo/listagem de hóspedes
`TecnicoManutencao`	`Funcionario`	`especialidade` e `registar_reparo()`
`QuartoSimples`	`Quarto`	tipo fixo "Simples"
`QuartoLuxo`	`Quarto`	tipo fixo "Luxo"

Exemplo
Em `Gerente(Funcionario)`, o construtor faz `super().__init__(nome, idade, salario)`. A função `super()` dá acesso à classe-mãe, e assim o `Gerente` não repete o código que guarda o nome, a idade e o salário.
Cadeia completa: `Gerente → Funcionario → Pessoa`. Um gerente tem, por herança, o `nome` e a `idade` validados de `Pessoa` e o `salario` de `Funcionario`.

2.8.3. Herança múltipla

Na herança múltipla, uma classe herda de **duas ou mais** classes-mãe. Foi aplicada uma única vez, na classe `TecnicoRecepcao(Recepcionista, TecnicoManutencao)`, que acumula as duas funções. Como se explicou na secção 2.6, é necessário ter atenção à MRO e ao uso de `super()`.

Exemplo
Um objeto `TecnicoRecepcao` é, em simultâneo, um `Recepcionista`, um `TecnicoManutencao`, um `Funcionario` e uma `Pessoa`. A Figura 7 confirma-o com `isinstance`. Por isso aparece nas listas de técnicos em `registrar_reparo()` e também pode usar `registar_hospede()`, herdado de `Recepcionista`.

```
TecnicoRecepcao
  Recepcionista
  TecnicoManutencao
  Funcionario
  Pessoa
  ABC
  object

isinstance(t, Recepcionista): True
isinstance(t, TecnicoManutencao): True
isinstance(t, Funcionario): True
isinstance(t, Pessoa): True
```

Figura 7 – MRO e verificações isinstance de um TecnicoRecepcao (execução real)

2.8.4. Polimorfismo

Polimorfismo ("muitas formas") é a capacidade de objetos de classes diferentes responderem à mesma chamada de método de maneiras diferentes. Em Python, resulta da **sobreposição de métodos** (*overriding*) combinada com a tipagem dinâmica: basta o objeto ter o método para ele poder ser chamado. No projeto há três exemplos claros.

Exemplo 1 – mostrar_informacoes() nas pessoas
O método `mostrar_informacoes()` é declarado abstrato em `Pessoa` e depois implementado de forma diferente em `Hospede`, `Funcionario`, `Gerente`, `Recepcionista`, `TecnicoManutencao` e `TecnicoRecepcao`. O `Gerente` mostra o bónus e o total; o `Recepcionista` mostra o turno; o `TecnicoManutencao` mostra a especialidade; o `TecnicoRecepcao` mostra o setor. A mesma chamada, `f.mostrar_informacoes()`, produz resultados diferentes (Figura 8).

```
Gerente: Carla | Idade: 45 | Salário Base: 2500€ | Bónus: 500€ | Total: 3000€
Recepcionista: João (ID: 1) | Idade: 29 | Turno: tarde | Salário: 1100€
Técnico de Manutenção: Sérgio | Idade: 38 | Especialidade: Canalização | Salário: 1300€
Técnico de Receção: Beatriz (ID: 9) | Setor: RH | Especialidade: Elétrica | Salário: 1250€
```

Figura 8 – Polimorfismo: a mesma chamada, quatro comportamentos diferentes

Exemplo 2 – gerar_relatorio()
Em `Gerente.gerar_relatorio(funcionarios)`, o ciclo `for f in funcionarios: f.mostrar_informacoes()` funciona para uma lista com objetos de tipos diferentes. O código do relatório **nunca precisa de ser alterado** quando se cria um novo tipo de funcionário, desde que este tenha `mostrar_informacoes()`. É este o grande benefício do polimorfismo: extensibilidade.

Exemplo 3 – registar_reparo()
O método `registar_reparo()` existe em `TecnicoManutencao` e é **redefinido** em `TecnicoRecepcao`. Em `registrar_reparo()` (main.py), a linha `tecnicos[escolha - 1].registar_reparo(descricao)` não sabe qual é o tipo do técnico escolhido; o Python decide em tempo de execução. Para o `TecnicoManutencao` aparece "Reparo registado por …"; para o `TecnicoRecepcao`, "Reparo registado na Receção (Setor …) por …" (Figura 13).

Exemplo 4 – quartos
Na lista `quartos` convivem objetos `QuartoSimples` e `QuartoLuxo`. `listar_quartos()` chama `quarto.mostrar_informacoes()` em todos, sem verificar o tipo.

2.8.5. Classes abstratas

Uma **classe abstrata** é uma classe que serve de modelo para outras e que não pode ser instanciada diretamente. Em Python é criada ao herdar de `ABC` e ao declarar pelo menos um método com `@abstractmethod`. O projeto tem duas: `Pessoa` e `Quarto`, ambas com o método abstrato `mostrar_informacoes()`.

Exemplo

Ao tentar executar `Pessoa()` ou `Quarto()`, o Python lança um `TypeError`, como se vê na Figura 9. Já `Hospede` e `QuartoLuxo`, por implementarem o método abstrato, são classes concretas e podem ser instanciadas.

```
Pessoa(): TypeError -> Can't instantiate abstract class Pessoa without an implementation for
abstract method 'mostrar_informacoes'
Quarto(): TypeError -> Can't instantiate abstract class Quarto without an implementation for
abstract method 'mostrar_informacoes'
```

Figura 9 – Tentativa de instanciar classes abstratas (execução real)

2.8.6. Encapsulamento e propriedades

Encapsulamento é esconder os detalhes internos de um objeto e controlar o acesso aos seus dados. No projeto isso é feito com atributos "protegidos" (`_nome`, `_idade`, `_ocupado`, `_preco_diaria`, `_turno`, `_quarto`, `_especialidade`) e com `@property` / `@x.setter`. As propriedades validam os valores:

- `Pessoa.idade` – não aceita valores negativos;
- `Quarto.preco_diaria` – não aceita preços negativos;
- `Recepcionista.turno` – só aceita manhã, tarde ou noite;
- `Hospede.quarto` – só aceita quartos livres e marca-os como ocupados.

Exemplo
A propriedade `Hospede.quarto` mostra o poder do setter: ao fazer `Hospede("Rui", 28, 2, q)` com um quarto já ocupado, o setter recusa a atribuição e o hóspede fica com `quarto = None`. Numa solução com atributos públicos, esta regra poderia ser facilmente quebrada em qualquer parte do código.

```
ValueError -> Preço da diária não pode ser negativo!
ValueError -> A idade não pode ser negativa!

turno guardado: noite
Turno inválido! Mantenha: manhã, tarde ou noite.
turno depois de valor inválido: manhã

ocupado após atribuição: True | conta: 400
Aviso: O quarto selecionado já se encontra ocupado!
segundo hóspede tem quarto? None
Checkout realizado com sucesso e quarto libertado!
ocupado após checkout: False
```

Figura 10 – Validação com setters e associação hóspede–quarto (execução real)

2.8.7. Módulos

Um **módulo** é um ficheiro Python que pode ser importado por outros. Permite dividir um programa grande em partes pequenas, reutilizáveis e fáceis de manter. O projeto tem cinco módulos de classes e um programa principal (Tabela 3). As importações usadas são, por exemplo, `from pessoa import Pessoa` (em `hospede.py` e `funcionario.py`), `from funcionario import Recepcionista, TecnicoManutencao` (em `tecnico_recepcao.py`) e `from quarto import QuartoSimples, QuartoLuxo` (em `main.py`).

2.8.8. Estruturas de dados

Foram utilizadas as seguintes estruturas de dados:

- **Listas:** `quartos`, `hospedes` e `funcionarios` guardam os objetos do hotel; uma lista literal `["manhã", "tarde", "noite"]` serve para validar o turno; e `lista_hospedes` é passada como argumento a `Recepcionista`. As operações usadas incluem `append`, `remove`, `len`, pesquisa sequencial e indexação (`tecnicos[escolha - 1]`).
- **Compreensão de lista:** `[f for f in funcionarios if isinstance(f, TecnicoManutencao)]` cria a lista de técnicos de forma compacta.
- **Expressão geradora:** `next((f for f in funcionarios if isinstance(f, Gerente)), None)` obtém o primeiro gerente sem criar uma lista intermédia.
- **Cadeias de caracteres (strings):** com os métodos `isdigit()`, `lower()` e a multiplicação `"-" * 35`.

Não foram necessários dicionários nem tuplos, pois as pesquisas são feitas sobre listas pequenas.

2.8.9. Laços de repetição

Tabela 5 – Laços de repetição usados no projeto

Laço	Onde	Para quê
`while True` + `break`	Menu principal (`main.py`)	Repetir o menu até o utilizador escolher 0
`for` simples	`listar_quartos`, `listar_hospedes`, `Gerente.gerar_relatorio`, `Recepcionista.listar_hospedes`	Percorrer todos os elementos de uma lista
`for` com `break`	`criar_hospede`, `fazer_checkout`	Procurar um elemento e parar quando é encontrado
`for` com validação	`criar_quarto`, `criar_funcionario`	Verificar duplicados (número do quarto, ID)
`for` + `enumerate`	`registrar_reparo`	Numerar os técnicos a partir de 1

2.8.10. Condicionais, validação e outras construções

O programa recorre extensivamente a estruturas condicionais (`if / elif / else`) para validar dados e decidir o fluxo. Outras construções relevantes são: as **f-strings** (`f"Quarto: {self.numero} | ..."`) para formatar texto; o **formato numérico** `:.2f` para euros; o **operador ternário** (`"Sim" if self.ocupado else "Não"`); as funções `isinstance` e `hasattr` para testar tipos e atributos; os **retornos antecipados** (`return` logo após uma mensagem de erro), que evitam ifs aninhados; e **funções** para dividir o `main.py` em tarefas pequenas.

2.8.11. Quadro-resumo

Tabela 6 – Quadro-resumo dos conceitos e respetivos exemplos

Conceito	Exemplo principal	Ficheiro
Herança simples	`class Gerente(Funcionario)`	funcionario.py

Conceito	Exemplo principal	Ficheiro
Herança múltipla	`class TecnicoRecepcao(Recepcionista, TecnicoManutencao)`	tecnico_recepcao.py
Polimorfismo	`f.mostrar_informacoes()` em `gerar_relatorio`	funcionario.py
Classe abstrata	`class Pessoa(ABC)` com `@abstractmethod`	pessoa.py
Classe abstrata	`class Quarto(ABC)` com `@abstractmethod`	quarto.py
Encapsulamento / property	`@idade.setter` com `ValueError`	pessoa.py
Módulos	`from pessoa import Pessoa`	hospede.py
Listas	`quartos = []`, `.append`, `.remove`	main.py
Laço `while`	`while True:` do menu	main.py
Laço `for`	`for quarto in quartos:`	main.py
`super()`	`super().__init__(nome, idade)`	hospede.py, funcionario.py, quarto.py

2.9. Testes e resultados

Para verificar o funcionamento, foram executados cenários completos através do menu e, em paralelo, pequenos testes às classes, sem menu. Nas figuras seguintes, as linhas em que aparece `Escolha uma opção:` seguidas de um valor mostram o que o utilizador escreveu; o texto "[... menu principal ...]" substitui a listagem repetida do menu, para poupar espaço.

2.9.1. Gestão de quartos

Neste cenário criam-se dois quartos (um simples e um de luxo), tenta-se repetir um número de quarto, listam-se os quartos e introduz-se um valor inválido (letras) como número de quarto. O sistema responde corretamente em todos os casos (ver Tabela 7); a listagem obtida mostra "Quarto: 101 | Tipo: Simples | Preço/Dia: 45.0€ | Ocupado: Não" e "Quarto: 102 | Tipo: Luxo | Preço/Dia: 120.5€ | Ocupado: Não".

2.9.2. Hóspedes, contas e checkout

Regista-se uma hóspede no quarto 101 e depois tenta-se alojar outro no mesmo quarto, o que é recusado. O segundo hóspede é alojado no quarto 102. A listagem mostra as contas (3 × 45 € = 135 €; 2 × 120,50 € = 241 €). Após o checkout da primeira hóspede (feito com o nome em minúsculas, que o programa aceita), o quarto 101 passa a livre (Figura 11).

```
[... menu principal ...]
Escolha uma opção: 1

--- CRIAR QUARTO ---
Número do quarto: 101
1 - Simples
2 - Luxo
Escolha o tipo: 1
Preço por dia (€): 45
Quarto criado com sucesso!
[... menu principal ...]
Escolha uma opção: 1

--- CRIAR QUARTO ---
```

```
Número do quarto: 102
Escolha o tipo: 2
Preço por dia (€): 120.5
Quarto criado com sucesso!
[... menu principal ...]
Escolha uma opção: 3

--- REGISTAR HÓSPEDE ---
Nome: Ana Silva
Idade: 34
Dias de estadia: 3

Quartos Existentes:
Quarto: 101 | Tipo: Simples | Preço/Dia: 45.0€ | Ocupado: Não
Quarto: 102 | Tipo: Luxo | Preço/Dia: 120.5€ | Ocupado: Não

Número do quarto para alojar: 101
Hóspede registado com sucesso!
[... menu principal ...]
Escolha uma opção: 3

--- REGISTAR HÓSPEDE ---
Nome: Rui Costa
Idade: 28
Dias de estadia: 2

(lista de quartos existentes)

Número do quarto para alojar: 101
O quarto selecionado já está ocupado!
[... menu principal ...]
Escolha uma opção: 3

--- REGISTAR HÓSPEDE ---
Nome: Rui Costa
Idade: 28
Dias de estadia: 2

(lista de quartos existentes)

Número do quarto para alojar: 102
Hóspede registado com sucesso!
[... menu principal ...]
Escolha uma opção: 4

--- LISTA DE HÓSPEDES ---
Hóspede: Ana Silva | Idade: 34 | Dias de Estadia: 3 | Quarto: 101
Valor Total da Conta: 135.00€
-----------------------------------
Hóspede: Rui Costa | Idade: 28 | Dias de Estadia: 2 | Quarto: 102
Valor Total da Conta: 241.00€
-----------------------------------
[... menu principal ...]
Escolha uma opção: 2

--- LISTA DE QUARTOS ---
Quarto: 101 | Tipo: Simples | Preço/Dia: 45.0€ | Ocupado: Sim
Quarto: 102 | Tipo: Luxo | Preço/Dia: 120.5€ | Ocupado: Sim
[... menu principal ...]
Escolha uma opção: 5

--- CHECKOUT ---
Nome do hóspede: ana silva
Valor total a pagar: 135.00€
Checkout realizado com sucesso e quarto libertado!
[... menu principal ...]
Escolha uma opção: 4
```

```
--- LISTA DE HÓSPEDES ---
Hóspede: Rui Costa | Idade: 28 | Dias de Estadia: 2 | Quarto: 102
Valor Total da Conta: 241.00€
---------------------------------
[... menu principal ...]
Escolha uma opção: 2

--- LISTA DE QUARTOS ---
Quarto: 101 | Tipo: Simples | Preço/Dia: 45.0€ | Ocupado: Não
Quarto: 102 | Tipo: Luxo | Preço/Dia: 120.5€ | Ocupado: Sim
[... menu principal ...]
Escolha uma opção: 0
A encerrar o sistema do hotel... Até breve!
```

Figura 11 – Execução: registo de hóspedes, contas e checkout

2.9.3. Funcionários e relatório

Criam-se os quatro tipos de funcionário. Observam-se três validações: o turno "madrugada" é rejeitado e o recepcionista fica com o turno "manhã"; o ID repetido é recusado; e o relatório é gerado pelo gerente, listando polimorficamente todos os funcionários (Figura 12).

```
[... menu principal ...]
Escolha uma opção: 6

--- CRIAR FUNCIONÁRIO ---
1 - Gerente
2 - Recepcionista
3 - Técnico de Manutenção
4 - Técnico de Receção
Escolha uma opção: 1
Nome: Carla Mendes
Idade: 45
Salário (€): 2500
Bónus (€): 500
Funcionário criado com sucesso!
[... menu principal ...]
Escolha uma opção: 6

--- CRIAR FUNCIONÁRIO ---
Escolha uma opção: 2
Nome: João Pires
Idade: 29
Salário (€): 1100
ID do funcionário: 7
Turno (manhã/tarde/noite): madrugada
Turno inválido! Mantenha: manhã, tarde ou noite.
Funcionário criado com sucesso!
[... menu principal ...]
Escolha uma opção: 6

--- CRIAR FUNCIONÁRIO ---
Escolha uma opção: 2
Nome: Teresa Dias
Idade: 41
Salário (€): 1150
ID do funcionário: 7
Já existe um funcionário com esse ID!
[... menu principal ...]
Escolha uma opção: 6

--- CRIAR FUNCIONÁRIO ---
Escolha uma opção: 3
Nome: Sérgio Lopes
Idade: 38
```

```
Salário (€): 1300
Especialidade: Canalização
Funcionário criado com sucesso!
[... menu principal ...]
Escolha uma opção: 6

--- CRIAR FUNCIONÁRIO ---
Escolha uma opção: 4
Nome: Beatriz Neves
Idade: 31
Salário (€): 1250
ID do funcionário: 9
Setor: RH
Especialidade: Elétrica
Funcionário criado com sucesso!
[... menu principal ...]
Escolha uma opção: 7

--- RELATÓRIO DO HOTEL ---

--- RELATÓRIO DE FUNCIONÁRIOS (GERADO PELO GERENTE) ---
Gerente: Carla Mendes | Idade: 45 | Salário Base: 2500.0€ | Bónus: 500.0€ | Total: 3000.0€
Recepcionista: João Pires (ID: 7) | Idade: 29 | Turno: manhã | Salário: 1100.0€
Técnico de Manutenção: Sérgio Lopes | Idade: 38 | Especialidade: Canalização | Salário: 1300.0€
Técnico de Receção: Beatriz Neves (ID: 9) | Setor: RH | Especialidade: Elétrica | Salário: 1250.0€
[... menu principal ...]
Escolha uma opção: 0
A encerrar o sistema do hotel... Até breve!
```

Figura 12 – Execução: criação de funcionários e relatório do gerente

Quando não existe nenhum gerente, o programa não falha: apresenta diretamente os dados de cada funcionário, como se pôde verificar ao criar apenas um técnico de manutenção e pedir o relatório ("Técnico de Manutenção: Sérgio Lopes | Idade: 38 | Especialidade: Canalização | Salário: 1300.0€").

2.9.4. Registo de reparos

Em primeiro lugar, tenta-se registar um reparo sem técnicos, e o programa avisa. Depois criam-se um técnico de manutenção e um técnico de receção, e regista-se um reparo em cada um. As mensagens são diferentes porque o método `registar_reparo()` foi redefinido (Figura 13).

```
[... menu principal ...]
Escolha uma opção: 8

--- REGISTAR REPARO ---
Descrição do reparo: Torneira
Não existem técnicos registados no sistema!
[... menu principal ...]
Escolha uma opção: 6

--- CRIAR FUNCIONÁRIO ---
1 - Gerente
2 - Recepcionista
3 - Técnico de Manutenção
4 - Técnico de Receção
Escolha uma opção: 3
Nome: Sérgio Lopes
Idade: 38
Salário (€): 1300
Especialidade: Canalização
Funcionário criado com sucesso!
[... menu principal ...]
Escolha uma opção: 6
```

```
--- CRIAR FUNCIONÁRIO ---
Escolha uma opção: 4
Nome: Beatriz Neves
Idade: 31
Salário (€): 1250
ID do funcionário: 9
Setor: RH
Especialidade: Elétrica
Funcionário criado com sucesso!
[... menu principal ...]
Escolha uma opção: 8

--- REGISTAR REPARO ---
Descrição do reparo: Lâmpada fundida no quarto 101
1 - Sérgio Lopes (TecnicoManutencao)
2 - Beatriz Neves (TecnicoRecepcao)
Escolha o técnico: 2
Reparo registado na Receção (Setor RH) por Beatriz Neves: Lâmpada fundida no quarto 101
[... menu principal ...]
Escolha uma opção: 8

--- REGISTAR REPARO ---
Descrição do reparo: Torneira a pingar
1 - Sérgio Lopes (TecnicoManutencao)
2 - Beatriz Neves (TecnicoRecepcao)
Escolha o técnico: 1
Reparo registado por Sérgio Lopes: Torneira a pingar
[... menu principal ...]
Escolha uma opção: 0
A encerrar o sistema do hotel... Até breve!
```

Figura 13 – Execução: registo de reparos (polimorfismo)

2.9.5. Quadro de casos de teste

Tabela 7 – Casos de teste realizados

Nº	Teste	Resultado esperado	Obtido
1	Criar quarto 101 (simples, 45 €)	Quarto criado	Passou
2	Criar outro quarto com o número 101	"Já existe um quarto com esse número!"	Passou
3	Número de quarto "abc"	"Número inválido!"	Passou
4	Tipo de quarto 3	"Tipo inválido!"	Passou
5	Registar hóspede com idade 0	"Idade inválida!"	Passou
6	Alojar hóspede em quarto ocupado	"O quarto selecionado já está ocupado!"	Passou
7	Conta de 3 dias a 45 €	135,00 €	Passou
8	Checkout com nome em minúsculas	Hóspede encontrado; quarto libertado	Passou
9	Turno "madrugada"	Aviso e turno "manhã"	Passou
10	ID de funcionário repetido	"Já existe um funcionário com esse ID!"	Passou
11	Criar técnico de receção (herança múltipla)	Funcionário criado sem erros	Passou (após correção)
12	Relatório sem gerente	Lista direta de funcionários	Passou
13	Reparo sem técnicos registados	"Não existem técnicos registados no sistema!"	Passou
14	Escolher técnico fora do intervalo	"Opção inválida!"	Passou
15	Instanciar `Pessoa` e `Quarto`	`TypeError` (classes abstratas)	Passou
16	Preço e idade negativos nas classes	`ValueError`	Passou
17	Opção inválida no menu principal	"Opção inválida! Tente novamente."	Passou

2.10. Verificação face ao enunciado

A Tabela 8 compara os métodos sugeridos no enunciado com o que foi implementado. O enunciado indica que se podem acrescentar funções ou classes sempre que necessário.

Tabela 8 – Cumprimento das funções sugeridas no enunciado

Classe	Sugestão do enunciado	Estado
`Pessoa`	`__init__`, `nome` (property e setter), `idade` (property e setter, sem negativos), `mostrar_informacoes` abstrato	Cumprido
`Hospede`	`__init__` com `dias_estadia` e `quarto=None`, `quarto` (property e setter só com quarto livre), `atribuir_quarto`, `calcular_conta`, `mostrar_informacoes`, `checkout`	Cumprido
`Gerente`	`__init__`, `mostrar_informacoes` (salário + bónus), `gerar_relatorio`	Cumprido
`Recepcionista`	`__init__` com `id_func` e turno, `turno` (property e setter com valores válidos), `mostrar_informacoes`, `registar_hospede`, `listar_hospedes`	Cumprido
`TecnicoManutencao`	`__init__`, `especialidade` (property), `mostrar_informacoes`, `registar_reparo`	Cumprido
`TecnicoRecepcao`	`__init__` com setor e especialidade, `registar_hospede` (herdado), `registar_reparo` (redefinido), `mostrar_informacoes` (redefinido)	Cumprido
`Quarto`	`__init__(numero, tipo, preco_diaria)`, `ocupado` (property), `preco_diaria` (property e setter sem negativos), `mostrar_informacoes`	Cumprido
Requisitos técnicos	Herança simples e múltipla, polimorfismo, classes abstratas, módulos, estruturas de dados e laços	Cumprido

Há duas pequenas diferenças em relação ao texto do enunciado, ambas intencionais: os métodos usam a grafia de Portugal (`registar_*` em vez de `registrar_*`), e o construtor de `TecnicoRecepcao` recebe também a `idade`, que é obrigatória para inicializar `Pessoa`.

2.11. Dificuldades encontradas e soluções

Tabela 9 – Principais dificuldades e como foram resolvidas

Dificuldade	Solução
`super()` em herança múltipla chamava o construtor errado e provocava `TypeError` ao criar um `TecnicoRecepcao`.	Compreender a MRO e inicializar diretamente `Funcionario.__init__`, atribuindo no próprio construtor os atributos específicos.
Garantir que um quarto não era atribuído a dois hóspedes.	Colocar a regra no setter de `Hospede.quarto` (e reforçá-la no menu), usando o estado `ocupado` do quarto.
Evitar que o programa terminasse com erro ao converter texto em número.	Validar com `isdigit()` antes de converter com `int()`.
Decidir quem emite o relatório.	Procurar um `Gerente` na lista com `next(...)` e delegar nele `gerar_relatorio`; se não existir, listar diretamente.

Dificuldade	Solução
Evitar IDs duplicados em funcionários de tipos diferentes.	Usar `hasattr(f, "id_func")` na verificação de duplicados.

CAPÍTULO III — ANÁLISE DO PERCURSO PESSOAL

Este capítulo reúne as principais aprendizagens técnicas obtidas com o desenvolvimento do projeto.

3.1. Aprendizagens técnicas

Antes deste trabalho, os conceitos de herança, polimorfismo e classes abstratas eram, para a equipa, definições teóricas. Ao aplicá-los a um problema concreto, foi possível perceber **para que servem**:

- A **herança** evita repetição de código: o nome, a idade e as suas validações escrevem-se uma só vez, em `Pessoa`, e são reutilizados por todas as pessoas do hotel.
- As **classes abstratas** funcionam como um contrato, que obriga as subclasses a implementarem certos métodos e impede a criação de objetos que não fazem sentido, como uma "pessoa" genérica.
- O **polimorfismo** torna o código mais simples e extensível: o relatório do gerente funciona para qualquer tipo de funcionário, incluindo tipos que ainda não existam.
- A **herança múltipla** é poderosa mas exige cuidado: o erro do `super()` mostrou que é essencial perceber a MRO.
- As **propriedades** (`@property`) permitem manter os atributos protegidos e validados, sem complicar a forma como os objetos são usados.
- A divisão em **módulos** torna o projeto mais organizado e facilita o trabalho em equipa.

CAPÍTULO IV — CONCLUSÃO

O projeto teve como objetivo desenvolver, em Python, um sistema de gestão de hotel que aplicasse corretamente herança simples e múltipla, polimorfismo, classes abstratas e módulos, seguindo o modelo de classes do enunciado. Considera-se que o objetivo foi **atingido**: o sistema permite criar e listar quartos de dois tipos, registar hóspedes e associá-los a quartos livres, calcular contas, efetuar checkouts, criar quatro tipos de funcionário, emitir um relatório e registar reparos.

Do ponto de vista técnico, o trabalho resultou numa solução com **dez classes** distribuídas por **cinco módulos** e um programa principal, com duas classes abstratas (`Pessoa` e `Quarto`), uma hierarquia de herança de três níveis (`Pessoa → Funcionario → Gerente/Recepcionista/TecnicoManutencao`) e um caso de herança múltipla (`TecnicoRecepcao`). O polimorfismo está presente na apresentação de informação de pessoas e quartos, no relatório do gerente e no registo de reparos. Todas as classes validam os seus dados através de propriedades, e o programa principal complementa essa proteção com validações de entrada.

Os testes realizados mostraram que o programa responde adequadamente a dados corretos e incorretos. O problema encontrado com o `super()` na classe `TecnicoRecepcao` foi, talvez, a aprendizagem mais valiosa, porque obrigou a compreender como funciona a MRO em vez de apenas memorizar a sintaxe da herança

múltipla. Em suma, o projeto permitiu consolidar os fundamentos da programação orientada a objetos.